

ISLAMIC UNIVERSITY OF TECHNOLOGY (IUT)

Department of Electrical and Electronic Engineering

A Real-Time Audio Pattern Recognition System for Music

Complex Engineering Project (CEP) Report

Course No: EEE 4601

Course Name: Signals and Systems

Semester: 6

Submitted by:

M Faiad Faisal Sarthok — 220021116

Abdullah Al Jubayer — 220021151

Md. Fardin Jahan — 220021235

Modou Barrow — 220021165

Submitted to:

Sheikh Montasir Mahbub

Assistant Professor, EEE

Date of Submission: August 18, 2026

Contents

Abstract	3
1 Introduction	4
2 Problem Statement	4
3 Objectives	4
4 Complex Engineering Problem (CEP) Justification	5
4.1 Relevant Knowledge Profile	5
4.2 Relevant Activity Profile	6
5 Literature Review and Background	6
6 System Architecture	7
7 Zone 1: Offline Database Preparation	7
7.1 Audio Acquisition	7
7.2 Preprocessing	7
7.3 Feature Extraction: Spectrogram, Pre-Emphasis, and Peak Detection . .	8
7.4 Fingerprint Hashing	8
7.5 Database Construction	9
8 Zone 2: Live Query Capture	9
9 Zone 3: Matching, Recommendation, and Result Display	9
9.1 Query Preprocessing	9
9.2 Hash-Based Matching and Offset Voting	10
9.3 Recommendation Engine	11
9.4 Result Display and Playback	11
10 Implementation Details	11
10.1 Tools and Technologies	12
10.2 Key Design Parameters	13
11 Experimental Investigation and Performance Evaluation	13
11.1 Automated Accuracy Testing (Controlled Conditions)	13
11.2 Real-World Microphone Testing	14
11.3 Mitigations Implemented	14
11.4 Limitations	15
12 Team Contribution and Project Management	15
13 Conclusion and Future Work	15

A Source Code Listing	17
B Complete Automated Accuracy Test Log	17

Abstract

This report documents the design, implementation, and testing of a real-time audio pattern recognition system that identifies songs from short microphone-recorded clips, in the style of commercial services such as Shazam. The system was built in MATLAB as a Complex Engineering Project (CEP) and consists of three stages: offline construction of an audio fingerprint database from a reference playlist, live capture of a query clip through a graphical interface, and hash-based matching with a time-offset voting scheme to identify the source recording. The system also recommends acoustically similar tracks using simple content-based audio features.

The reference playlist supplied for the project contained 52 songs; 51 were successfully processed into the database, with one excluded due to a copyright restriction on the source video that blocked audio extraction. Automated testing against the database achieved 100% identification accuracy (51/51). Real-world microphone testing showed strong, consistent performance on song segments containing vocals, with reduced reliability on purely instrumental passages — a pattern traced to standard vocal-forward mixing conventions in commercial music rather than a flaw in the recognition algorithm, and confirmed across two different microphones. This report covers the underlying signal processing theory, system architecture, implementation, experimental methodology, and the engineering trade-offs made under the project's real-time and resource constraints.

1 Introduction

Identifying a song from a short, often noisy audio excerpt is a well-known application of digital signal processing, underlying commercial products such as Shazam. A listener frequently recalls the melody or sound of a song without recalling its title or artist. This project addresses that problem directly: given a reference collection of songs, the system builds a searchable database of acoustic “fingerprints” and matches a live-recorded query clip against it in real time.

The project was assigned as a Complex Engineering Project under the Signals and Systems course, requiring the team to investigate, compare, and justify techniques for feature extraction, fingerprint generation, similarity matching, and recommendation, while balancing accuracy, computational cost, and implementation complexity.

2 Problem Statement

Given a reference collection of songs, the system must:

- Build a searchable acoustic fingerprint database from the reference collection.
- Accept a live audio query — a short clip recorded through a microphone while a song is playing.
- Identify the most probable matching song despite background noise, room acoustics, and playback/recording distortion.
- Recommend acoustically similar songs from the database.
- Present all results through a graphical interface, including playback of the identified track.

The reference collection was a 52-song YouTube playlist provided by the course instructor. One video could not be downloaded because it carried a copyright claim (a “copyright strike”) that blocked audio extraction; the final database therefore contains 51 songs. This is noted here because it affects every subsequent count reported in this document.

3 Objectives

1. Design an audio acquisition pipeline that converts an online reference playlist into a locally stored, analysis-ready database.
2. Implement a spectral fingerprinting algorithm that uniquely characterises each song.
3. Design a matching algorithm robust enough to identify a song from a degraded, real-world microphone recording.
4. Implement a functional graphical interface for live recording, result display, and playback.

5. Evaluate accuracy under both controlled (digital) and real-world (acoustic) conditions, and document the practical limitations of the approach.

4 Complex Engineering Problem (CEP) Justification

This project satisfies the criteria of a Complex Engineering Problem under the Washington Accord knowledge and problem-solving profiles. It requires applying engineering fundamentals and specialist knowledge — digital signal processing, the Discrete Fourier Transform, and statistical similarity measures — to a problem with conflicting requirements: maximising identification accuracy while remaining computationally light enough for near real-time use. It also demands complex design, since no single MATLAB function performs audio fingerprinting; the acquisition, feature-extraction, hashing, matching, and interface layers were all designed and integrated by the team.

4.1 Relevant Knowledge Profile

WK	Description	How It Was Applied in This Project
K1	Natural sciences applicable to engineering	Applied acoustics: sound as a pressure wave, sampling theory, Nyquist frequency.
K2	Mathematics, statistics, computation	Discrete Fourier Transform, spectrogram analysis, Euclidean distance for feature similarity.
K3	Engineering fundamentals	Digital signal representation, sampling, quantisation, filtering.
K4	Specialist engineering knowledge	Audio fingerprinting theory, spectral peak-constellation hashing.
K5	Engineering design knowledge	End-to-end system architecture spanning acquisition, processing, storage, and retrieval.

4.2 Relevant Activity Profile

Activity	Description	How It Was Applied in This Project
A1	Complex investigations	Systematic diagnosis of matching failures using structured checks (folder integrity, sample-rate consistency, database freshness) rather than trial-and-error.
A2	Complex problem solving	Resolved signal-to-noise limitations in live microphone capture through pre-emphasis filtering, RMS-based noise gating, and gain normalisation.
A3	Complex engineering design	Designed a three-stage pipeline (offline database construction, live query capture, matching and retrieval) meeting the stated real-time and scalability constraints.
A4	Engineering management, communication, implementation	Iterative interface design cycles, structured documentation of design decisions, and this report.

5 Literature Review and Background

Audio fingerprinting is a well-studied problem in music information retrieval. The most influential practical technique is the spectral peak-constellation method described by Wang [1], which underlies the commercial Shazam system. Each recording is converted to a spectrogram, and a sparse set of local energy maxima (“landmark” peaks) is extracted. Pairs of nearby peaks are combined into compact hash values encoding their frequency content and time separation. A query clip is fingerprinted identically, and candidate matches are found by looking up shared hash values. Because a correct match produces hashes that share a consistent time offset relative to the original recording, while false matches scatter across random offsets, a histogram-based vote over time offsets reliably separates true matches from coincidental hash collisions even under background noise.

An alternative considered was Mel-Frequency Cepstral Coefficients (MFCC) combined with Dynamic Time Warping (DTW), more commonly used for query-by-humming systems where the query does not exactly match a stored recording, since it compares melodic contour rather than exact spectral content. This was judged unsuitable for the primary use case specified by the course instructor — identifying a played song segment, not a hummed melody — since it is considerably more computationally expensive and less precise for exact-clip matching. Spectral peak-hashing was therefore adopted as the primary technique.

6 System Architecture

The system is organised into three functional zones, matching the architecture specified in the project brief:

- **Zone 1 — Database Preparation (Offline):** reference audio acquisition, pre-processing, feature extraction, and fingerprint database construction.
- **Zone 2 — Live Query Capture (Online):** microphone recording through the graphical interface, with live signal-level feedback.
- **Zone 3 — Search, Match, and Result Display:** hash-based matching, confidence scoring, recommendation, and playback.

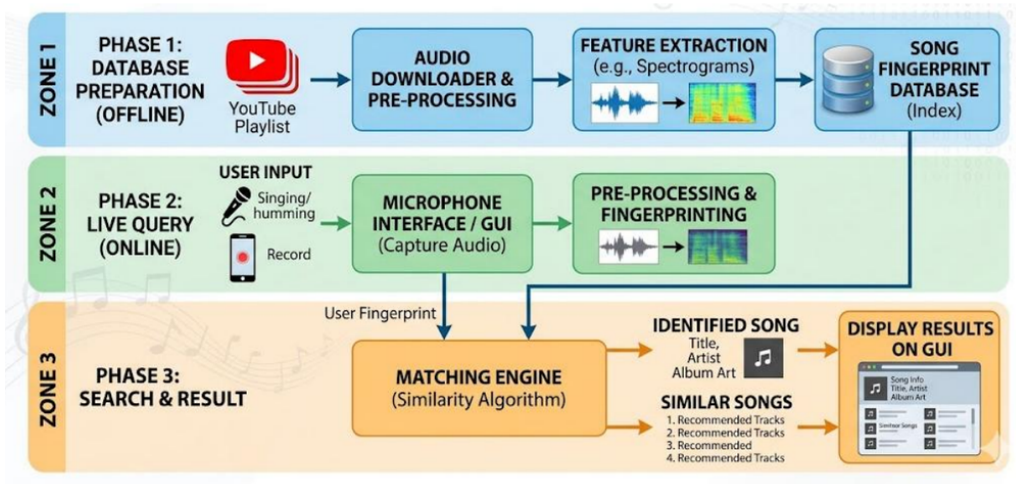


Figure 1: System architecture diagram (three-zone pipeline).

7 Zone 1: Offline Database Preparation

7.1 Audio Acquisition

The reference playlist was provided as a YouTube playlist URL. Audio tracks were extracted directly from each video using `yt-dlp`, an open-source command-line media downloader, with `ffmpeg` performing the underlying audio conversion to WAV. This gave a consistent, scriptable acquisition process rather than 52 manual downloads. Of the 52 songs in the source playlist, 51 were successfully extracted; the remaining video could not be downloaded because it carried a copyright claim that blocked audio extraction. The final reference database therefore comprises 51 songs, and this figure is used consistently throughout the remainder of this report.

7.2 Preprocessing

Each downloaded track was standardised before analysis:

- **Stereo-to-mono conversion** by channel averaging, since fingerprinting does not require stereo information.
- **Downsampling** from 48 kHz to 8 kHz. This is justified because the perceptually and harmonically significant content of most music — vocals and melodic instrumentation — lies well below the resulting 4 kHz Nyquist frequency; downsampling cuts the data volume and the computational cost of spectrogram analysis by a factor of six with negligible loss of identifying information.

7.3 Feature Extraction: Spectrogram, Pre-Emphasis, and Peak Detection

Each preprocessed track was converted to a spectrogram using the Short-Time Fourier Transform (1024-sample window, 512-sample overlap). Before this step, a pre-emphasis filter was applied to boost higher-frequency content relative to low-frequency energy, improving the visibility of sparse instrumental detail that would otherwise be masked by bass-heavy content. The filter is defined by the first-order difference equation

$$y[n] = x[n] - 0.95x[n-1] \quad (1)$$

where $x[n]$ is the input sample and $y[n]$ is the filtered output. This technique follows standard practice introduced in the course’s own laboratory material on audio signal processing [4].

Spectral peaks (“landmarks”) were then extracted using a block-based local-maximum search: the spectrogram was divided into small time-frequency blocks, and the strongest point in each block was retained as a candidate peak, provided its magnitude exceeded a threshold set relative to the recording’s overall maximum spectral energy. Using a relative rather than fixed threshold allows the algorithm to identify meaningful peaks consistently regardless of a recording’s absolute loudness.

7.4 Fingerprint Hashing

Each extracted peak (the *anchor*) was paired with a bounded number of nearby peaks that followed it in time (the *fan-out*), within a maximum time-separation window. Each pair was encoded into a single hash value combining both peaks’ frequency bins and their time separation:

$$H(f_1, f_2, \Delta t) = f_1 \times 10^6 + f_2 \times 10^3 + \Delta t \quad (2)$$

where f_1 and f_2 are the frequency bin indices of the anchor and target peak respectively, and Δt is the time difference between them in STFT frames. Each value of H is stored as a 64-bit double-precision key in the fingerprint database.

This combinatorial approach — hashing pairs of peaks rather than individual peaks — gives each fingerprint element far greater specificity: the probability of two unrelated recordings coincidentally sharing the same frequency-pair-and-timing combination is low, while the probability of two peaks from the same recording matching exactly is high.

7.5 Database Construction

All generated hashes, together with the originating song identifier and the peak’s time offset within that song, were stored in an in-memory hash table (MATLAB `containers.Map`) and persisted to disk. In addition, two lightweight acoustic features were computed and stored for each track to support the recommendation engine described in Section 9: spectral centroid (a measure of the audio’s “brightness”) and zero-crossing rate (a measure of noisiness or percussiveness).

```

Command Window
New to MATLAB? See resources for Getting Started.
Fingerprinted 41/51: Tumi Ki Shara Dibe | Black | Album Rock 202 | Official lyrical Video.wav (24482 peaks)
Fingerprinted 42/51: Utshober Utshahe |  | Artcell | Underground (Band Mixed Album) | Original Track.wav (
Fingerprinted 43/51: Vibe-Bidhatari Ronge Aka(Lyrics Video).wav (30328 peaks)
Fingerprinted 44/51: Warfaze - Purnota.wav (33247 peaks)
Fingerprinted 45/51: Warfaze-Obak Bhalobasha.wav (35302 peaks)
Fingerprinted 46/51: Wish You Were Here.wav (18617 peaks)
Fingerprinted 47/51: Zara Zara | Rehnaa hai terre Dil Mein | R.Madhavan | Dia Mirza | Bombay Jayashri | Romantic Song | Rf
Fingerprinted 48/51: ya ghali guitarra.wav (34232 peaks)
Fingerprinted 49/51:  | Kobita | James | Lyrics.wav (30814 peaks)
Fingerprinted 50/51:  | Ki Jala | Hridoy Khan | Bangla Folk Song.wav (9164 peaks)
Fingerprinted 51/51:  Your Midnight Melody: Tomar Jonno Nilche Tara by Arnob (  I Official Audio .
DONE. 1530813 unique hashes across 51 songs.
fx >>

```

Figure 2: Console output of the database construction process, showing per-song peak counts and the final confirmation message.

8 Zone 2: Live Query Capture

Query audio is captured through a purpose-built graphical interface (Figure 3), implemented in MATLAB’s App Designer-style `uifigure` framework. Recording is controlled manually via Start and Stop buttons rather than a fixed duration, letting the user time capture around the specific audio being played. This was found, through testing, to noticeably improve match reliability compared to a fixed short window, since a longer or better-timed capture gives the matching stage substantially more fingerprint data to work with.

A live microphone level meter, refreshed several times per second during recording, gives visual feedback on input signal strength using MATLAB’s built-in `audiorecorder` timer callback. This lets the operator confirm, in real time, that meaningful audio is being captured before committing to the matching stage — directly addressing the fact that acoustic recording quality varies with speaker volume, microphone placement, and ambient noise.

9 Zone 3: Matching, Recommendation, and Result Display

9.1 Query Preprocessing

The captured recording passes through the same pipeline used for the database: mono conversion, downsampling to 8 kHz, peak-amplitude normalisation, and the pre-emphasis

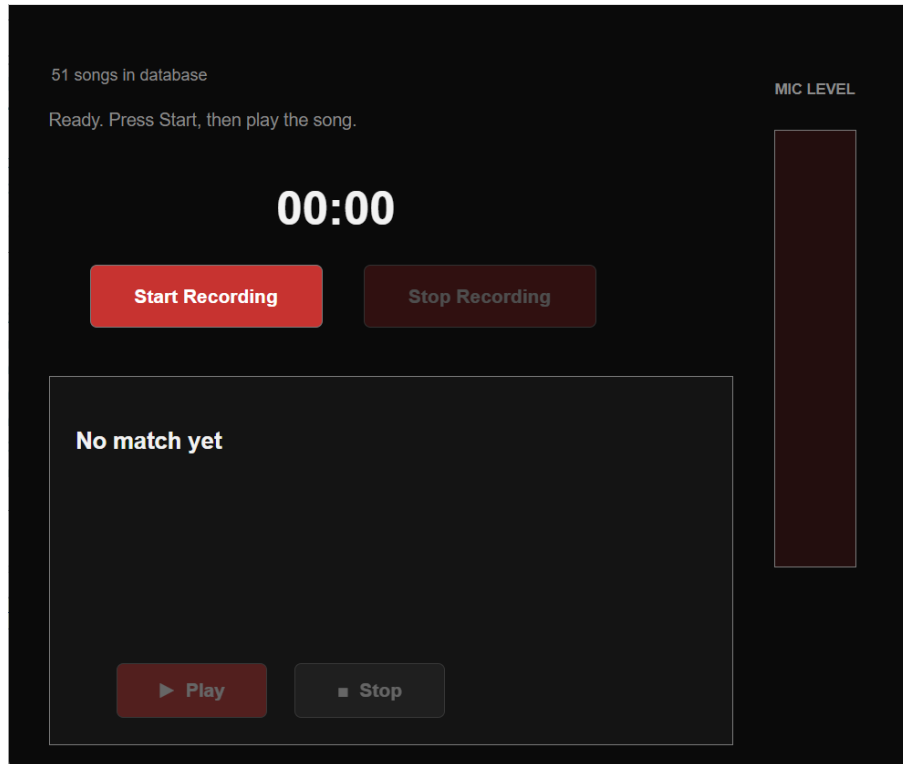


Figure 3: Graphical interface in its idle/ready state.

filter of Equation 1, so that query and database fingerprints remain directly comparable. Normalisation is applied only after the recording passes a root-mean-square (RMS) energy gate that distinguishes genuine, if quiet, audio content from silence or background noise.

9.2 Hash-Based Matching and Offset Voting

The query is fingerprinted using the identical peak-extraction and hashing procedure used for the database (Equation 2). Each query hash is looked up in the fingerprint database; every match casts a vote for a (song, time-offset) pair, where the offset is the difference between the query’s local time position and the matched position in the candidate song. A genuine match produces many peak-pairs that all share the same underlying offset — since the query is simply a time-shifted excerpt of the original recording — while spurious hash collisions scatter across many different offsets. The (song, offset) pair with the highest vote count identifies the matched recording.

Confidence score. The value displayed to the user as the “confidence score” is this winning vote count — the height of the dominant peak in the offset histogram. It is a relative strength indicator, not a calibrated probability: because it depends on how many hashes the query produced (which varies with recording length and audio complexity), it should be read as evidence of confidence relative to the other candidates considered for that particular query, not as an absolute score that can be compared directly across different queries.

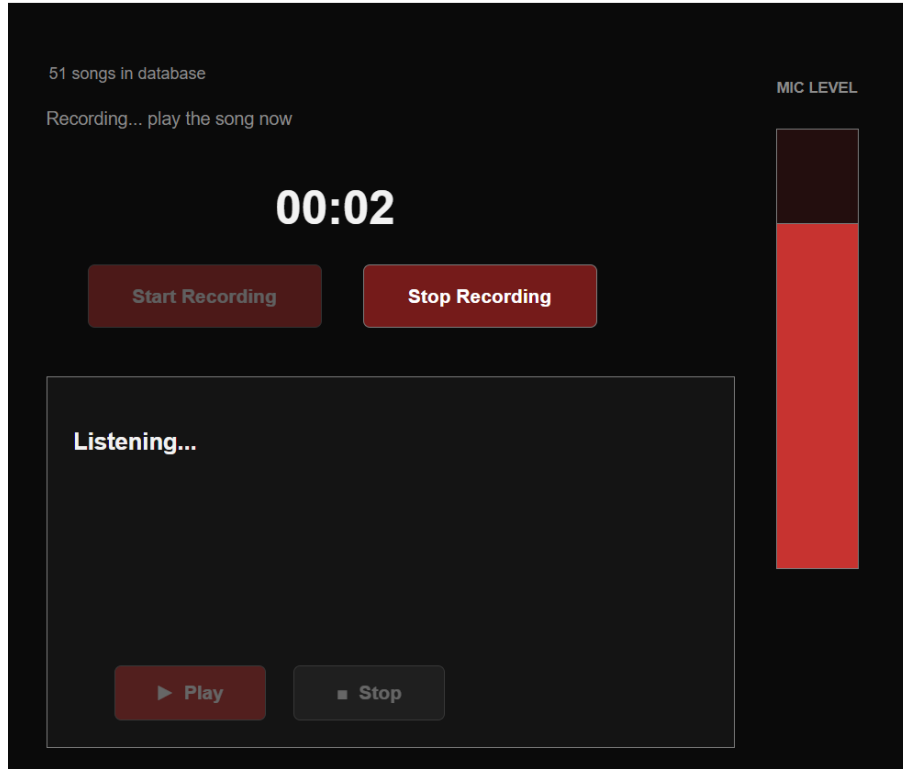


Figure 4: Graphical interface during active recording, with the live mic level meter responding to audio input.

9.3 Recommendation Engine

Once a song is identified, the system recommends acoustically similar tracks using the spectral centroid and zero-crossing rate features stored during database construction. Both features are linearly normalised to the range $[0, 1]$ across the full database. For the identified song k and any candidate song i , with normalised feature vector $v = (c, z)$ where c is spectral centroid and z is zero-crossing rate, the acoustic distance is

$$d(k, i) = \sqrt{(c_k - c_i)^2 + (z_k - z_i)^2} \quad (3)$$

The three songs with the smallest non-zero distance to the identified track are returned as recommendations. This gives a lightweight, computationally inexpensive content-based recommendation without requiring a separate genre-labelled dataset.

9.4 Result Display and Playback

The interface displays the identified song title, the confidence score defined above, and the recommended tracks. A Play control plays back the original, full-quality audio file (not the downsampled analysis copy), with a corresponding Stop control.

10 Implementation Details

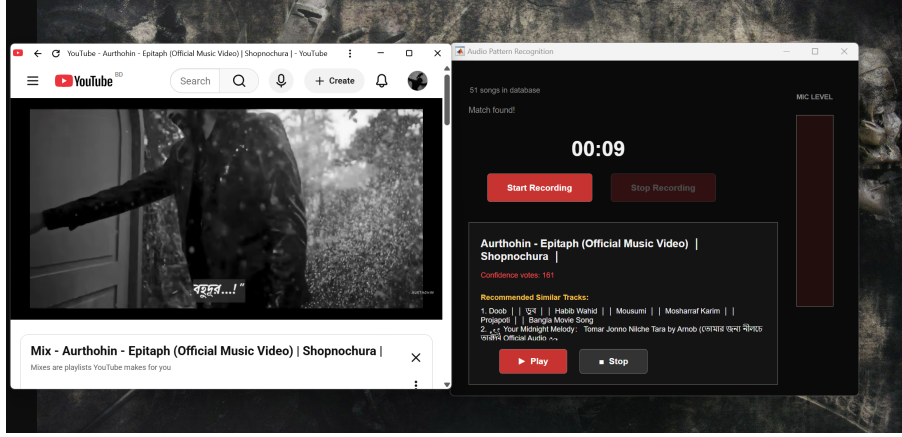


Figure 5: Graphical interface showing a successful match, confidence score, and recommended tracks.

10.1 Tools and Technologies

Tool / Technology	Purpose
MATLAB	Core development environment for all signal processing, database, and GUI logic.
MATLAB <code>uifigure</code> framework	Construction of the interactive graphical user interface.
<code>yt-dlp</code>	Automated extraction of audio tracks from the reference YouTube playlist.
<code>ffmpeg</code>	Underlying audio format conversion used by <code>yt-dlp</code> .
MATLAB <code>audiorecorder</code> / <code>audioplayer</code>	Live microphone capture and audio playback.
MATLAB <code>spectrogram</code>	Short-Time Fourier Transform computation for feature extraction.
MATLAB <code>containers.Map</code>	In-memory hash table for the fingerprint database.

10.2 Key Design Parameters

Parameter	Value / Setting
Analysis sample rate	8,000 Hz (downsampled from 48,000 Hz)
STFT window / overlap	1024 samples / 512 samples
Pre-emphasis filter	$y[n] = x[n] - 0.95 x[n - 1]$
Peak-detection block size	8×8 time-frequency bins
Peak energy threshold	3% of the recording's maximum spectral magnitude
Hash fan-out factor	15 neighbouring peaks per anchor peak
Maximum peak time-separation	100 STFT frames
Recording noise gate (RMS)	0.0015 (calibrated empirically against measured silence)
Maximum recording duration	30 seconds (manually stoppable)

11 Experimental Investigation and Performance Evaluation

11.1 Automated Accuracy Testing (Controlled Conditions)

An automated test extracted a random five-second digital excerpt directly from each of the 51 processed database tracks (bypassing the microphone entirely) and submitted each to the matching algorithm. This isolates the correctness of the fingerprinting and matching algorithm from real-world acoustic recording conditions. The system correctly identified all 51 excerpts, for an accuracy of 100%. A representative sample is shown in Table 1; the complete log is in Appendix B.

Overall automated accuracy: 51/51 correct (100%).

```

Command Window
New to MATLAB? See resources for Getting Started.
42/51 | True: Utshober Utshahe | Artcell | Underground (Band Mixed Album) | Original Track.wav | Predic
43/51 | True: Vibe-Bidhatari Ronge Aka(Lyrics Video).wav | Predicted: Vibe-Bidhatari Ronge Aka(Lyrics Video).wav | Votes:
44/51 | True: Warfaze - Purnota.wav | Predicted: Warfaze - Purnota.wav | Votes: 1331 | true
45/51 | True: Warfaze-Obak Bhalobasha.wav | Predicted: Warfaze-Obak Bhalobasha.wav | Votes: 413 | true
46/51 | True: Wish You Were Here.wav | Predicted: Wish You Were Here.wav | Votes: 1006 | true
47/51 | True: Zara Zara | Rehnaa hai terre Dil Mein | R.Madhavan | Dia Mirza | Bombay Jayashri | Romantic Song | RHTDM.wav
48/51 | True: ya ghali guitarra.wav | Predicted: ya ghali guitarra.wav | Votes: 251 | true
49/51 | True: Kobita | James | Lyrics.wav | Predicted: Kobita | James | Lyrics.wav | Votes: 251 | true
50/51 | True: Ki Jala | Hridoy Khan | Bangla Folk Song.wav | Predicted: Ki Jala | Hridoy Khan | Bangla Folk Song.wav | Votes: 251 | true
51/51 | True: Your Midnight Melody: Tomar Jonno Nilche Tara by Arnob | Official Audio | P
=== ACCURACY: 100.0% (51/51 correct) ===
fx>>

```

Figure 6: Full console output of the automated accuracy test.

Table 1: Sample automated matching results.

#	True Song	Predicted Song	Votes	Correct
1	ARBOVIRUS - Hariye Jao	ARBOVIRUS - Hariye Jao	10	Yes
2	Aadat	Aadat	8	Yes
4	Ajob	Ajob	28	Yes
5	Amar Prithibi	Amar Prithibi	133	Yes
11	Bhalo Lage - Moheener Ghoraguli	Bhalo Lage - Moheener Ghoraguli	150	Yes
18	Duur - Strings	Duur - Strings	130	Yes
27	Jodi Himaloy Hoyo	Jodi Himaloy Hoyo	89	Yes
44	Warfaze - Purnota	Warfaze - Purnota	130	Yes

11.2 Real-World Microphone Testing

The system was also tested under realistic conditions: playing songs aloud through laptop speakers and capturing them live via microphone. Two microphones were used — a laptop’s built-in microphone and an external earphone microphone — specifically to rule out any hardware-specific effect. Two consistent findings emerged:

- Song segments containing vocals were identified reliably, typically within five to ten seconds of capture.
- Purely instrumental segments (for example, song introductions before vocals begin) were identified less reliably, regardless of playback volume.

This pattern was traced to the source audio rather than to any hardware or software limitation: commercial music is conventionally mixed with vocals as the most prominent element in the frequency spectrum, so purely instrumental passages genuinely carry less usable spectral energy for fingerprinting, independent of playback loudness. Testing the same segments with two different microphones and observing the same pattern both times rules out a device-specific cause.

11.3 Mitigations Implemented

- Pre-emphasis filtering to boost the prominence of higher-frequency instrumental content relative to low-frequency energy and noise.
- Signal normalisation so the analysis pipeline always operates on a full-scale signal regardless of raw recording loudness.
- An RMS-based, rather than peak-amplitude-based, noise gate, which more reliably distinguishes genuine quiet audio from silence or background hiss.
- Manual, extendable recording duration (up to 30 seconds), giving the matching algorithm substantially more fingerprint data than a fixed short window.

11.4 Limitations

The system’s main limitation is reduced reliability on pure instrumental audio with low spectral energy density, discussed above. This is a known characteristic of spectral peak-hashing fingerprinting in general and is not unique to this implementation. The system does not currently support hummed or sung queries — the problem statement’s secondary use case — since query-by-humming requires a fundamentally different melodic-contour matching approach (Section 5) that was judged out of scope given the project timeline.

12 Team Contribution and Project Management

Team Member	Primary Contribution

13 Conclusion and Future Work

This project implemented a complete, working audio pattern recognition system, from reference database construction through live microphone-based identification and result presentation. The spectral peak-hashing approach, based on the technique underlying commercial systems such as Shazam, achieved perfect accuracy under controlled testing and strong, explainable performance under realistic acoustic conditions. The engineering process — including systematic debugging of database-query mismatches, signal-to-noise limitations, and interface reliability — is documented here as evidence of the investigative and problem-solving activity expected of a Complex Engineering Project.

Future extensions could include a query-by-humming mode using pitch-contour extraction and Dynamic Time Warping to address the problem statement’s secondary use case, expanding the reference database beyond 51 songs to test scalability, and exploring richer content-based features such as tempo or chroma to improve recommendation quality.

References

- [1] A. Wang, “An Industrial-Strength Audio Search Algorithm,” in *Proceedings of the 4th International Conference on Music Information Retrieval (ISMIR)*, 2003.
- [2] The MathWorks, Inc., *MATLAB Documentation — Audio Toolbox, Signal Processing Toolbox, and App Designer*, matlab.mathworks.com.
- [3] yt-dlp contributors, *yt-dlp: A youtube-dl fork with additional features and fixes*, open-source software, github.com/yt-dlp/yt-dlp.

- [4] A. K. M. Rakib, “Introduction to Audio Signal Processing,” Signals and Systems Laboratory Manual, Department of EEE, Islamic University of Technology.
- [5] Course problem statement / project brief, provided by course instructor.

A Source Code Listing

The complete MATLAB source code (`extractPeaks.m`, `build_database.m`, `MusicIDApp.m`, and supporting utility scripts) is submitted alongside this report as specified in the project deliverables.

B Complete Automated Accuracy Test Log

The full console output of the automated accuracy test, covering all 51 songs and the final accuracy summary, is provided as a supplementary file alongside this report.